# 10. Errors Deeply: Wrap, Is, As

## 10. Errors Deeply: Wrap, Is, As

**Phase:** Phase 2 — Core language depth
**Estimated time:** 1–2 days
**Slug:** `10-errors-deep`

### Learning objectives

- Wrap errors with context using fmt.Errorf and %w
- Use errors.Is and errors.As for inspection
- Choose sentinel vs custom error types
- Design error messages for operators and callers

### Prerequisites

Complete prior numbered lessons. Type every example yourself.

### Wrapping

```
if err != nil {
    return fmt.Errorf("load config %s: %w", path, err)
}
```

`%w` preserves the chain for `errors.Is` / `errors.As`.

### Sentinel errors

```
var ErrNotFound = errors.New("not found")
```

Good for stable, package-level conditions. Do not compare error strings.

### Typed errors

```
type ValidationError struct{ Field string }
func (e *ValidationError) Error() string { return e.Field + " invalid" }

var ve *ValidationError
if errors.As(err, &ve) { /* ... */ }
```

### Guideline

Return errors; log at the edge (main/handler). Avoid logging and returning the same error repeatedly.

### Common mistakes (especially from Python instincts)

- Comparing `err.Error() == "..."`
- Using `%v` instead of `%w` when wrapping
- Creating unique error strings instead of sentinels

## Exercises

Work in `exercises/10-errors-deep/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

**Do not open `solutions/10-errors-deep/` until you have an honest failing attempt.**

## Self-check

1. Difference between errors.Is and errors.As?
2. When prefer sentinel vs typed error?
3. Where should you log errors?

## Further reading

- https://go.dev/blog/go1.13-errors
- https://pkg.go.dev/errors